Лекция №5.
OpenMP. Подробнее.



Григорьев Василий Васильевич

2023



Директива threadprivate

- Директива threadprivate указывает, что переменные из списка должны быть размножены с тем, чтобы каждая нить имела свою локальную копию.

#pragma omp threadprivate(список)

Пример threadprivate

```c
#include <stdio.h>
#include <omp.h>
int n;
#pragma omp threadprivate(n)
int main(int argc, char *argv[])
{
 omp_set_num_threads(8);
 int num;
 n=1;
#pragma omp parallel private (num)
 {
 num=omp_get_thread_num();
 printf("Значение n на нити %d (на входе): %d\n", num, n);
 /* Присвоим переменной n номер текущей нити */
 n=omp_get_thread_num();
 printf("Значение n на нити %d (на выходе): %d\n", num, n);
 }
 printf("Значение n (середина): %d\n", n);
#pragma omp parallel private (num)
 {
 num=omp_get_thread_num();
 printf("Значение n на нити %d (ещё раз): %d\n", num, n);
 }
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычис
Значение n на нити 1 (на входе): 0
Значение n на нити 1 (на выходе): 1
Значение n на нити 5 (на входе): 0
Значение n на нити 5 (на выходе): 5
Значение n на нити 2 (на входе): 0
Значение n на нити 2 (на выходе): 2
Значение n на нити 0 (на входе): 1
Значение n на нити 0 (на выходе): 0
Значение n на нити 3 (на входе): 0
Значение n на нити 3 (на выходе): 3
Значение n на нити 6 (на входе): 0
Значение n на нити 6 (на выходе): 6
Значение n на нити 4 (на входе): 0
Значение n на нити 4 (на выходе): 4
Значение n на нити 7 (на входе): 0
Значение n на нити 7 (на выходе): 7
Значение n (середина): 0
Значение n на нити 0 (ещё раз): 0
Значение n на нити 6 (ещё раз): 6
Значение n на нити 7 (ещё раз): 7
Значение n на нити 1 (ещё раз): 1
Значение n на нити 2 (ещё раз): 2
Значение n на нити 4 (ещё раз): 4
Значение n на нити 3 (ещё раз): 3
Значение n на нити 5 (ещё раз): 5
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычис
```

Глобальная переменная n объявлена как threadprivate переменная. Значение переменной n выводится в четырёх разных местах. Первый раз все нити выведут значение своей копии переменной n в начале параллельной области, и это значение будет равно 1 на нити-мастере и 0 на остальных нитях. Далее с помощью функции omp_get_thread_num() все нити присвоят переменной n свой порядковый номер и выведут это значение. Затем в последовательной области будет ещё раз выведено значение переменной n, которое окажется равным порядковому номеру нити-мастера, то есть 0. В последний раз значение переменной n выводится в новой параллельной области, причём значение каждой локальной копии должно сохраниться.

copyin

- Если необходимо переменную, объявленную как threadprivate, инициализировать значением размножаемой переменной из нити-мастера, то на входе в параллельную область можно использовать опцию copyin. Если значение локальной переменной или переменной, объявленной как threadprivate, необходимо переслать от одной нити всем, работающим в данной параллельной области, для этого можно использовать опцию copyprivate директивы single.

Пример copyin

```
#include <stdio.h>
int n;
#pragma omp threadprivate(n)
int main(int argc, char *argv[])
{
 n=1;
#pragma omp parallel copyin(n)
 {
 printf("Значение n: %d\n", n);
 }
}
```

```
vasiliy@BEAST:/
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
Значение n: 1
vasiliy@BEAST:/
```

Глобальная переменная n определена как threadprivate. Применение опции copyin позволяет инициализировать локальные копии переменной n начальным значением нити-мастера. Все нити выведут значение n, равное 1.

Распределение работы

- OpenMP предлагает несколько вариантов распределения работы между запущенными нитями. Конструкции распределения работ в OpenMP не порождают новых нитей.

Низкоуровневое распараллеливание

- Все нити в параллельной области нумеруются последовательными целыми числами от 0 до N-1, где N — количество нитей, выполняющих данную область.
- Можно программировать на самом низком уровне, распределяя работу с помощью функций omp_get_thread_num() и omp_get_num_threads(), возвращающих номер нити и общее количество порождённых нитей в текущей параллельной области, соответственно.
- Вызов функции omp_get_thread_num() позволяет нити получить свой уникальный номер в текущей параллельной области.

Параллельные циклы

- Если в параллельной области встретился оператор цикла, то, согласно общему правилу, он будет выполнен всеми нитями текущей группы, то есть каждая нить выполнит все итерации данного цикла. Для распределения итераций цикла между различными нитями можно использовать директиву for.

#pragma omp for [опция [[,] опция]...]

Возможные опции

- private(список) – задаёт список переменных, для которых порождается локальная копия в каждой нити; начальное значение локальных копий переменных из списка не определено;
- firstprivate(список) – задаёт список переменных, для которых порождается локальная копия в каждой нити; локальные копии переменных инициализируются значениями этих переменных в нити-мастере;
- lastprivate(список) – переменным, перечисленным в списке, присваивается результат с последнего витка цикла;
- reduction(оператор:список) – задаёт оператор и список общих переменных; для каждой переменной создаются локальные копии в каждой нити; локальные копии инициализируются соответственно типу оператора (для аддитивных операций – 0 или его аналоги, для мультипликативных операций – 1 или её аналоги); над локальными копиями переменных после завершения всех итераций цикла выполняется заданный оператор; оператор это: – +, *, -, &, |, ^, &&, ||; порядок выполнения операторов не определён, поэтому результат может отличаться от запуска к запуску;

Возможные опции

- schedule(type[, chunk]) – опция задаёт, каким образом итерации цикла распределяются между нитями;
- collapse(n) — опция указывает, что n последовательных тесновложенных циклов ассоциируется с данной директивой; для циклов образуется общее пространство итераций, которое делится между нитями; если опция collapse не задана, то директива относится только к одному непосредственно следующему за ней циклу;
- ordered – опция, говорящая о том, что в цикле могут встречаться директивы ordered; в этом случае определяется блок внутри тела цикла, который должен выполняться в том порядке, в котором итерации идут в последовательном цикле;
- nowait – в конце параллельного цикла происходит неявная барьерная синхронизация параллельно работающих нитей: их дальнейшее выполнение происходит только тогда, когда все они достигнут данной точки; если в подобной задержке нет необходимости, опция nowait позволяет нитям, уже дошедшим до конца цикла, продолжить выполнение без синхронизации с остальными.

Важно!

- На вид параллельных циклов накладываются достаточно жёсткие ограничения. В частности, предполагается, что корректная программа не должна зависеть от того, какая именно нить какую итерацию параллельного цикла выполнит. Нельзя использовать побочный выход из параллельного цикла. Размер блока итераций, указанный в опции schedule, не должен изменяться в рамках цикла.

Важно!

- Формат параллельных циклов на языке Си упрощённо можно представить следующим образом:

```
for([целочисленный тип] i = инвариант цикла;
        i {,=,<=,>=} инвариант цикла;
        i {+,-}= инвариант цикла)
```

Эти требования введены для того, чтобы OpenMP мог при входе в цикл точно определить число итераций.

Если директива параллельного выполнения стоит перед гнездом циклов, завершающихся одним оператором, то директива действует только на самый внешний цикл

Нюанс

- Итеративная переменная распределяемого цикла по смыслу должна быть локальной, поэтому в случае, если она специфицирована общей, то она неявно делается локальной при входе в цикл. После завершения цикла значение итеративной переменной цикла не определено, если она не указана в опции lastprivate.

Пример

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
    int A[10], B[10], C[10], i, n;
    /* Заполним исходные массивы */
    for (i=0; i<10; i++)
    {
        A[i]=i; B[i]=2*i; C[i]=0;
    }
    #pragma omp parallel shared(A, B, C) private(i,
n)
    {
        /* Получим номер текущей нити */
        n=omp_get_thread_num();
        #pragma omp for
            for (i=0; i<10; i++)
            {
                C[i]=A[i]+B[i];
                printf("Нить %d сложила элементы с
номером %d\n", n, i);
            }
    }
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Выч
Нить 0 сложила элементы с номером 0
Нить 4 сложила элементы с номером 4
Нить 9 сложила элементы с номером 9
Нить 2 сложила элементы с номером 2
Нить 3 сложила элементы с номером 3
Нить 8 сложила элементы с номером 8
Нить 1 сложила элементы с номером 1
Нить 6 сложила элементы с номером 6
Нить 5 сложила элементы с номером 5
Нить 7 сложила элементы с номером 7
vasiliy@BEAST:/mnt/d/Dropbox/lol/Выч
```

В последовательной области инициализируются три исходных массива A, B, C. В параллельной области данные массивы объявлены общими. Вспомогательные переменные i и n объявлены локальными

Каждая нить присвоит переменной n свой порядковый номер. Далее с помощью директивы for (do) определяется цикл, итерации которого будут распределены между существующими нитями. На каждой i-ой итерации данный цикл сложит i-ые элементы массивов A и B и результат запишет в i-ый элемент массива C. Также на каждой итерации будет напечатан номер нити, выполнившей данную итерацию.

Опции schedule

В опции schedule параметр type задаёт следующий тип распределения итераций:

- **static** – блочно-циклическое распределение итераций цикла; размер блока – **chunk**. Первый блок из **chunk** итераций выполняет нулевая нить, второй блок — следующая и т.д. до последней нити, затем распределение снова начинается с нулевой нити. Если значение **chunk** не указано, то всё множество итераций делится на непрерывные куски примерно одинакового размера (конкретный способ зависит от реализации), и полученные порции итераций распределяются между нитями.
- **dynamic** – динамическое распределение итераций с фиксированным размером блока: сначала каждая нить получает **chunk** итераций (по умолчанию **chunk=1**), та нить, которая заканчивает выполнение своей порции итераций, получает первую свободную порцию из **chunk** итераций. Освободившиеся нити получают новые порции итераций до тех пор, пока все порции не будут исчерпаны. Последняя порция может содержать меньше итераций, чем все остальные.

Опции schedule

- **guided** – динамическое распределение итераций, при котором размер порции уменьшается с некоторого начального значения до величины **chunk** (по умолчанию **chunk=1**) пропорционально количеству ещё не распределённых итераций, делённому на количество нитей, выполняющих цикл. Размер первоначально выделяемого блока зависит от реализации. В ряде случаев такое распределение позволяет аккуратнее разделить работу и сбалансировать загрузку нитей. Количество итераций в последней порции может оказаться меньше значения **chunk**.
- **auto** – способ распределения итераций выбирается компилятором и/или системой выполнения. Параметр **chunk** при этом не задаётся.
- **runtime** – способ распределения итераций выбирается во время работы программы по значению переменной среды OMP_SCHEDULE. Параметр **chunk** при этом не задаётся.

Пример schedule

```c
#include <stdio.h>
#include <omp.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    int i;
    #pragma omp parallel private(i)
    {
        #pragma omp for schedule (static)
        //#pragma omp for schedule (static, 1)
        //#pragma omp for schedule (static, 2)
        //#pragma omp for schedule (dynamic)
        //#pragma omp for schedule (dynamic, 2)
        //#pragma omp for schedule (guided)
        //#pragma omp for schedule (guided, 2)
        for (i=0; i<10; i++)
        {
            printf("Нить %d выполнила итерацию %d\n", omp_get_thread_num(), i);
            sleep(1);
        }
    }
}
```

```
Нить 0 выполнила итерацию 0
Нить 7 выполнила итерацию 7
vasiliy@BEAST:/mnt/d/Dropbox/lol/
vasiliy@BEAST:/mnt/d/Dropbox/lol/
Нить 7 выполнила итерацию 7
Нить 6 выполнила итерацию 6
Нить 2 выполнила итерацию 2
Нить 4 выполнила итерацию 4
Нить 5 выполнила итерацию 5
Нить 1 выполнила итерацию 1
Нить 9 выполнила итерацию 9
Нить 8 выполнила итерацию 8
Нить 3 выполнила итерацию 3
Нить 0 выполнила итерацию 0
vasiliy@BEAST:/mnt/d/Dropbox/lol/
vasiliy@BEAST:/mnt/d/Dropbox/lol/
Нить 0 выполнила итерацию 0
Нить 2 выполнила итерацию 4
Нить 3 выполнила итерацию 6
Нить 1 выполнила итерацию 2
Нить 4 выполнила итерацию 8
Нить 2 выполнила итерацию 5
Нить 1 выполнила итерацию 3
Нить 0 выполнила итерацию 1
Нить 3 выполнила итерацию 7
Нить 4 выполнила итерацию 9
vasiliy@BEAST:/mnt/d/Dropbox/lol/
vasiliy@BEAST:/mnt/d/Dropbox/lol/
Нить 2 выполнила итерацию 7
Нить 3 выполнила итерацию 5
Нить 23 выполнила итерацию 0
Нить 15 выполнила итерацию 8
Нить 21 выполнила итерацию 9
Нить 10 выполнила итерацию 1
Нить 5 выполнила итерацию 3
vasiliy@BEAST:/mnt/d/Dropbox/lol/
vasiliy@BEAST:/mnt/d/Dropbox/lol/
Нить 20 выполнила итерацию 6
Нить 15 выполнила итерацию 8
Нить 3 выполнила итерацию 4
Нить 14 выполнила итерацию 0
Нить 1 выполнила итерацию 2
Нить 20 выполнила итерацию 7
Нить 3 выполнила итерацию 5
Нить 14 выполнила итерацию 1
Нить 15 выполнила итерацию 9
Нить 1 выполнила итерацию 3
vasiliy@BEAST:/mnt/d/Dropbox/lol/
vasiliy@BEAST:/mnt/d/Dropbox/lol/
Нить 3 выполнила итерацию 3
Нить 0 выполнила итерацию 0
Нить 21 выполнила итерацию 1
Нить 1 выполнила итерацию 2
Нить 10 выполнила итерацию 4
Нить 13 выполнила итерацию 5
Нить 6 выполнила итерацию 6
Нить 7 выполнила итерацию 7
Нить 15 выполнила итерацию 8
Нить 23 выполнила итерацию 9
vasiliy@BEAST:/mnt/d/Dropbox/lol/
vasiliy@BEAST:/mnt/d/Dropbox/lol/
Нить 18 выполнила итерацию 4
Нить 16 выполнила итерацию 8
Нить 19 выполнила итерацию 2
Нить 4 выполнила итерацию 0
Нить 22 выполнила итерацию 6
Нить 18 выполнила итерацию 5
Нить 16 выполнила итерацию 9
Нить 19 выполнила итерацию 3
Нить 4 выполнила итерацию 1
Нить 22 выполнила итерацию 7
vasiliy@BEAST:/mnt/d/Dropbox/lol/
```

Вот так получилось у автора, а у меня чуть по другому, или так же?

i	static	static, 1	static, 2	dynamic	dynamic, 2	guided	guided, 2
0	0	0	0	0	0	0	0
1	0	1	0	1	0	2	0
2	0	2	1	2	1	1	1
3	1	3	1	3	1	3	1
4	1	0	2	1	2	1	2
5	1	1	2	3	2	2	2
6	2	2	3	2	3	3	3
7	2	3	3	0	3	0	3
8	3	0	0	1	3	0	0
9	3	1	0	0	3	3	0

Таблица 1. Распределение итераций по нитям.

Обобщенный пример

```
#include <stdio.h>
#include <omp.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    int i;
    #pragma omp parallel private(i)
    {
        #pragma omp for schedule (static, 6)
        //#pragma omp for schedule (dynamic, 6)
        //#pragma omp for schedule (guided, 6)
        for (i=0; i<200; i++)
        {
            printf("Нить %d выполнила итерацию %d\n",
                omp_get_thread_num(), i);
            sleep(1);
        }
    }
}
```

Что? Как?

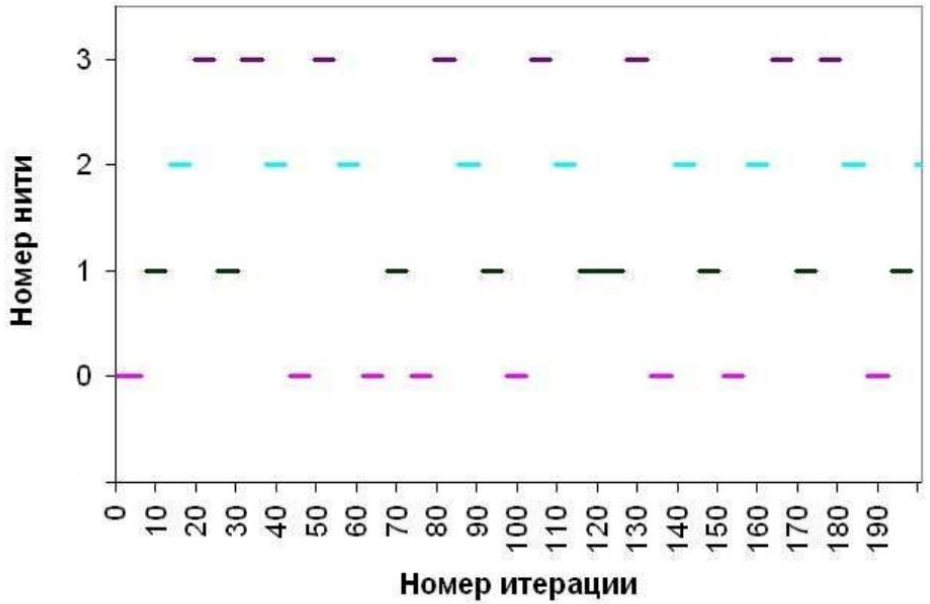


Рис.1b. Распределение итераций по нитям для (dynamic, 6)

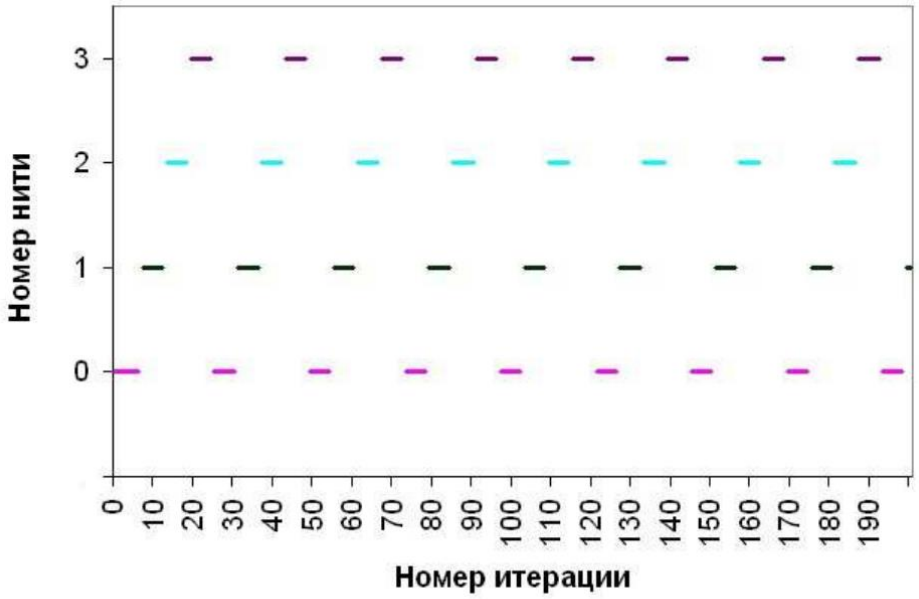


Рис.1a. Распределение итераций по нитям для (static, 6)

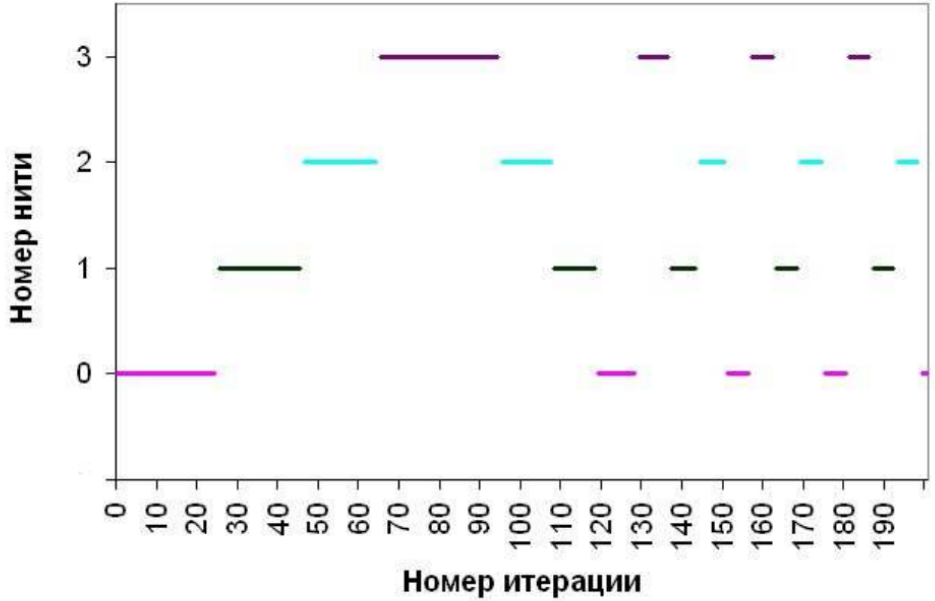


Рис.1c. Распределение итераций по нитям для (guided, 6)

Надо знать.

- Значение по умолчанию переменной OMP_SCHEDULE зависит от реализации. Если переменная задана неправильно, то поведение программы при задании опции runtime также зависит от реализации.
- Изменить значение переменной OMP_SCHEDULE из программы можно с помощью вызова функции omp_set_schedule().

```
void omp_set_schedule(omp_sched_t type, int chunk);
```

- При помощи вызова функции omp_get_schedule() пользователь может узнать текущее значение переменной OMP_SCHEDULE.

```
void omp_get_schedule(omp_sched_t* type, int* chunk);
```

Что?

- Допустимые значения констант описаны в файле omp.h (omp_lib.h). Как минимум, они должны включать для языка Си следующие варианты:

```
typedef enum omp_sched_t {
omp_sched_static = 1,
omp_sched_dynamic = 2,
omp_sched_guided = 3,
omp_sched_auto = 4
} omp_sched_t;
```

Параллельные секции

- Директива sections (sections ... end sections) используется для задания конечного (неитеративного) параллелизма.

#pragma omp sections [опция [[,] опция]...]

- Эта директива определяет набор независимых секций кода, каждая из которых выполняется своей нитью.

Возможные опции

- private(список) – задаёт список переменных, для которых порождается локальная копия в каждой нити; начальное значение локальных копий переменных из списка не определено;
- firstprivate(список) – задаёт список переменных, для которых порождается локальная копия в каждой нити; локальные копии переменных инициализируются значениями этих переменных в нити-мастере;
- lastprivate(список) – переменным, перечисленным в списке, присваивается результат, полученный в последней секции;
- reduction(оператор:список) – задаёт оператор и список общих переменных; для каждой переменной создаются локальные копии в каждой нити; локальные копии инициализируются соответственно типу оператора (для аддитивных операций – 0 или его аналоги, для мультипликативных операций – 1 или её аналоги); над локальными копиями переменных после завершения всех секций выполняется заданный оператор; оператор это: – +, *, -, &, |, ^, &&, ||; порядок выполнения операторов не определён, поэтому результат может отличаться от запуска к запуску;
- nowait – в конце блока секций происходит неявная барьерная синхронизация параллельно работающих нитей: их дальнейшее выполнение происходит только тогда, когда все они достигнут данной точки; если в подобной задержке нет необходимости, опция nowait позволяет нитям, уже дошедшим до конца своих секций, продолжить выполнение без синхронизации с остальными.

section

- Директива section задаёт участок кода внутри секции sections для выполнения одной нитью.

#pragma omp section

- Перед первым участком кода в блоке sections директива section не обязательна. Какие именно нити будут задействованы для выполнения какой секции, не специфицируется. Если количество нитей больше количества секций, то часть нитей для выполнения данного блока секций не будет задействована. Если количество нитей меньше количества секций, то некоторым (или всем) нитям достанется более одной секции.

Пример

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
    int n;
    #pragma omp parallel private(n)
    {
        n=omp_get_thread_num();
        #pragma omp sections
        {
            #pragma omp section
            {
                printf("Первая секция, процесс %d\n", n);
            }
            #pragma omp section
            {
                printf("Вторая секция, процесс %d\n", n);
            }
            #pragma omp section
            {
                printf("Третья секция, процесс %d\n", n);
            }
        }
        printf("Параллельная область, процесс %d\n", n);
    }
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/B
Вторая секция, процесс 4
Первая секция, процесс 8
Третья секция, процесс 18
Параллельная область, процесс 8
Параллельная область, процесс 22
Параллельная область, процесс 4
Параллельная область, процесс 16
Параллельная область, процесс 9
Параллельная область, процесс 21
Параллельная область, процесс 10
Параллельная область, процесс 11
Параллельная область, процесс 5
Параллельная область, процесс 1
Параллельная область, процесс 0
Параллельная область, процесс 13
Параллельная область, процесс 23
Параллельная область, процесс 2
Параллельная область, процесс 19
Параллельная область, процесс 12
Параллельная область, процесс 3
Параллельная область, процесс 6
Параллельная область, процесс 17
Параллельная область, процесс 15
Параллельная область, процесс 14
Параллельная область, процесс 20
Параллельная область, процесс 7
Параллельная область, процесс 18
vasiliy@BEAST:/mnt/d/Dropbox/lol/B
```

Сначала три нити, на которые распределились три секции section, выведут сообщение со своим номером, а потом все нити напечатают одинаковое сообщение со своим номером.

Пример с lastprivate

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
 int n=0;
#pragma omp parallel
 {
#pragma omp sections lastprivate(n)
 {
#pragma omp section
 {
 n=1;
 }
#pragma omp section
 {
 n=2;
 }
#pragma omp section
 {
 n=3;
 }
 }
 printf("Значение n на нити %d: %d\n",
 omp_get_thread_num(), n);
 }
 printf("Значение n в последовательной области: %d\n", n);
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислите
Значение n на нити 0: 3
Значение n на нити 18: 3
Значение n на нити 6: 3
Значение n на нити 9: 3
Значение n на нити 20: 3
Значение n на нити 22: 3
Значение n на нити 2: 3
Значение n на нити 7: 3
Значение n на нити 10: 3
Значение n на нити 16: 3
Значение n на нити 19: 3
Значение n на нити 13: 3
Значение n на нити 1: 3
Значение n на нити 14: 3
Значение n на нити 17: 3
Значение n на нити 3: 3
Значение n на нити 8: 3
Значение n на нити 12: 3
Значение n на нити 5: 3
Значение n на нити 21: 3
Значение n на нити 4: 3
Значение n на нити 15: 3
Значение n на нити 23: 3
Значение n на нити 11: 3
Значение n в последовательной области: 3
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислите
```

В данном примере опция lastprivate используется вместе с директивой sections. Переменная n объявлена как lastprivate переменная. Три нити, выполняющие секции section, присваивают своей локальной копии n разные значения. По выходе из области sections значение n из последней секции присваивается локальным копиям во всех нитях, поэтому все нити напечатают число 3. Это же значение сохранится для переменной n и в последовательной области.

Задачи (tasks)

- Директива task применяется для выделения отдельной независимой задачи.

#pragma omp task [опция [[,] опция]...]

- Текущая нить выделяет в качестве задачи ассоциированный с директивой блок операторов. Задача может выполняться немедленно после создания или быть отложенной на неопределённое время и выполняться по частям. Размер таких частей, а также порядок выполнения частей разных отложенных задач определяется реализацией.

Возможные опции

- **if(условие)** — порождение новой задачи только при выполнении некоторого условия; если условие не выполняется, то задача будет выполнена текущей нитью и немедленно;
- **untied** — опция означает, что в случае откладывания задача может быть продолжена любой нитью из числа выполняющих данную параллельную область; если данная опция не указана, то задача может быть продолжена только породившей её нитью;
- **default(private|firstprivate|shared|none)** – всем переменным в задаче, которым явно не назначен класс, будет назначен класс private, firstprivate или shared соответственно; none означает, что всем переменным в задаче класс должен быть назначен явно; в языке Си задаются только варианты shared или none;

Возможные опции

- private(список) – задаёт список переменных, для которых порождается локальная копия в каждой нити; начальное значение локальных копий переменных из списка не определено;
- firstprivate(список) – задаёт список переменных, для которых порождается локальная копия в каждой нити; локальные копии переменных инициализируются значениями этих переменных в нити-мастере;
- shared(список) – задаёт список переменных, общих для всех нитей.

taskwait

- Для гарантированного завершения в точке вызова всех запущенных задач используется директива taskwait.

```
#pragma omp taskwait
```

- Нить, выполнившая данную директиву, приостанавливается до тех пор, пока не будут завершены все ранее запущенные данной нитью независимые задачи.

Работает как барьер, но только для тасков.

Что еще?

- task очень похож на sections, но sections статичны, то есть количество sections задается при написании кода, тогда как задачи можно создавать в любое время и в любом количестве под контролем логики вашей программы.

Пример

```c
#include <stdio.h>
#include <omp.h>
int fib(int n)
{
  int i, j;
  if (n<2)
    return n;
  else
    {
      #pragma omp task shared(i) firstprivate(n)
      i=fib(n-1);

      #pragma omp task shared(j) firstprivate(n)
      j=fib(n-2);

      #pragma omp taskwait
      return i+j;
    }
}
```

```c
int main()
{
  int n = 10;

  omp_set_dynamic(0);
  omp_set_num_threads(24);

  #pragma omp parallel shared(n)
  {
    #pragma omp single
    printf ("fib(%d) = %d\n", n, fib(n));
  }
}
```

```
fib(10) = 55
```

Директива ordered

- Директивы ordered определяют блок внутри тела цикла, который должен выполняться в том порядке, в котором итерации идут в последовательном цикле.

```
#pragma omp ordered
```

- Блок операторов относится к самому внутреннему из объемлющих циклов, а в параллельном цикле должна быть задана опция ordered. Нить, выполняющая первую итерацию цикла, выполняет операции данного блока. Нить, выполняющая любую следующую итерацию, должна сначала дождаться выполнения всех операций блока всеми нитями, выполняющими предыдущие итерации. Может использоваться, например, для упорядочения вывода от параллельных нитей.

Пример ordered

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
 int i, n;
#pragma omp parallel private (i, n)
 {
 n=omp_get_thread_num();
#pragma omp for ordered
 for (i=0; i<5; i++)
 {
        printf("Нить %d, итерация %d\n", n,
i);
        #pragma omp ordered
        {
                printf("ordered: Нить %d,
итерация %d\n", n, i);
        }
 }
 }
}
```

Пример иллюстрирует применение директивы ordered и опции ordered. Цикл for помечен как ordered. Внутри тела цикла идут две выдачи – одна вне блока ordered, а вторая – внутри него. В результате первая выдача получается неупорядоченной, а вторая идёт в строгом порядке по возрастанию номера итерации.

```
vasiliy@BEAST:/mnt/d/Dropbox/
Нить 0, итерация 0
Нить 4, итерация 4
Нить 1, итерация 1
Нить 2, итерация 2
Нить 3, итерация 3
ordered: Нить 0, итерация 0
ordered: Нить 1, итерация 1
ordered: Нить 2, итерация 2
ordered: Нить 3, итерация 3
ordered: Нить 4, итерация 4
vasiliy@BEAST:/mnt/d/Dropbox/
```

Критические секции

- С помощью директив critical оформляется критическая секция программы.

  #pragma omp critical [(<имя критической секции>)]

- В каждый момент времени в критической секции может находиться не более одной нити. Если критическая секция уже выполняется какой-либо нитью, то все другие нити, выполнившие директиву для секции с данным именем, будут заблокированы, пока вошедшая нить не закончит выполнение данной критической секции. Как только работавшая нить выйдет из критической секции, одна из заблокированных на входе нитей войдет в неё. Если на входе в критическую секцию стояло несколько нитей, то случайным образом выбирается одна из них, а остальные заблокированные нити продолжают ожидание.

Критические секции

- Все неименованные критические секции условно ассоциируются с одним и тем же именем. Все критические секции, имеющие одно и тоже имя, рассматриваются единой секцией, даже если находятся в разных параллельных областях. Побочные входы и выходы из критической секции запрещены.

Пример critical

```c
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
    int n;
    omp_set_num_threads(8);
    #pragma omp parallel
    {
        #pragma omp critical
        {
            n=omp_get_thread_num();
            printf("Нить %d\n", n);
        }
    }
}
```

```
vasiliy@B
Нить 5
Нить 2
Нить 4
Нить 3
Нить 0
Нить 6
Нить 1
Нить 7
vasiliy@B
```

Эта директива используется чтобы предотвратить «race conditions» и «насильственные потоки» при выполнении сегмента кода один за другим.
Тем самым, код выполняется одним потоком на единицу времени (а не сразу много потоков выполняют одновременно).

Когда его использовать?

Использовать нужно тогда, когда:

- Часть кода использует данные, которые зависят от предыдущего состояния, или используют какие то общие (shared) данные между несколькими потоками;
- Чтобы избежать любых «race conditions»

Директива atomic

- Частым случаем использования критических секций на практике является обновление общих переменных. Например, если переменная sum является общей и оператор вида sum=sum+expr находится в параллельной области программы, то при одновременном выполнении данного оператора несколькими нитями можно получить некорректный результат. Чтобы избежать такой ситуации можно воспользоваться механизмом критических секций или специально предусмотренной для таких случаев директивой atomic.

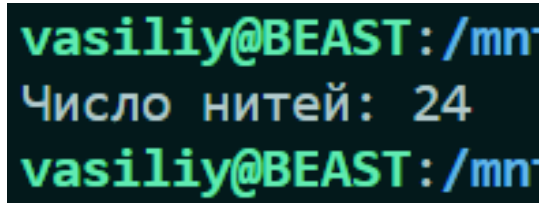
#pragma omp atomic

Директива atomic

- Данная директива относится к идущему непосредственно за ней оператору присваивания (на используемые в котором конструкции накладываются достаточно понятные ограничения), гарантируя корректную работу с общей переменной, стоящей в его левой части. На время выполнения оператора блокируется доступ к данной переменной всем запущенным в данный момент нитям, кроме нити, выполняющей операцию. Атомарной является только работа с переменной в левой части оператора присваивания, при этом вычисления в правой части не обязаны быть атомарными.

Пример использования atomic

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
 int count = 0;
#pragma omp parallel
 {
    #pragma omp atomic
        count++;
 }
 printf("Число нитей: %d\n", count);
}
```



В данном примере производится подсчет общего количества порожденных нитей. Для этого каждая нить увеличивает на единицу значение переменной count. Для того, чтобы предотвратить одновременное изменение несколькими нитями значения переменной, стоящей в левой части оператора присваивания, используется директива atomic.

Замки (locks)

- Один из вариантов синхронизации в OpenMP реализуется через механизм замков (locks). В качестве замков используются общие целочисленные переменные (размер должен быть достаточным для хранения адреса). Данные переменные должны использоваться только как параметры примитивов синхронизации.
- Замок может находиться в одном из трёх состояний: неинициализированный, разблокированный или заблокированный. Разблокированный замок может быть захвачен некоторой нитью. При этом он переходит в заблокированное состояние. Нить, захватившая замок, и только она может его освободить, после чего замок возвращается в разблокированное состояние.

Замки (locks)

- Есть два типа замков: простые замки и множественные замки. Множественный замок может многократно захватываться одной нитью перед его освобождением, в то время как простой замок может быть захвачен только однажды. Для множественного замка вводится понятие коэффициента захваченности (nesting count). Изначально он устанавливается в ноль, при каждом следующем захватывании увеличивается на единицу, а при каждом освобождении уменьшается на единицу. Множественный замок считается разблокированным, если его коэффициент захваченности равен нулю.
- Для инициализации простого или множественного замка используются соответственно функции omp_init_lock() и omp_init_nest_lock().

Что?

```
void omp_init_lock(omp_lock_t *lock);
void omp_init_nest_lock(omp_nest_lock_t *lock);
```

- После выполнения функции замок переводится в разблокированное состояние. Для множественного замка коэффициент захваченности устанавливается в ноль.
- Функции omp_destroy_lock() и omp_destroy_nest_lock() используются для переведения простого или множественного замка в неинициализированное состояние.

Что?

```
void omp_destroy_lock(omp_lock_t *lock);
void omp_destroy_nest_lock(omp_nest_lock_t *lock);
```

Для захватывания замка используются функции omp_set_lock() и omp_set_nest_lock().

```
void omp_set_lock(omp_lock_t *lock);
void omp_set_nest_lock(omp_nest_lock_t *lock);
```

Вызов этой функции освобождает простой замок, если он был захвачен вызвавшей нитью. Для множественного замка уменьшает на единицу коэффициент захваченности. Если коэффициент станет равен нулю, замок освобождается. Если после освобождения замка есть нити, заблокированные на операции, захватывающей данный замок, замок будет сразу же захвачен одной из ожидающих нитей.

Привет

```c
#include <stdio.h>
#include <omp.h>
#include <unistd.h>

omp_lock_t lock;
int main(int argc, char *argv[])
{
    int n;
    omp_init_lock(&lock);
    #pragma omp parallel private (n)
    {
        n=omp_get_thread_num();
        omp_set_lock(&lock);
        printf("Начало закрытой секции, нить %d\n", n);
        sleep(5);
        printf("Конец закрытой секции, нить %d\n", n);
        omp_unset_lock(&lock);
    }
    omp_destroy_lock(&lock);
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/B
Начало закрытой секции, нить 15
Конец закрытой секции, нить 15
Начало закрытой секции, нить 10
Конец закрытой секции, нить 10
Начало закрытой секции, нить 18
Конец закрытой секции, нить 18
Начало закрытой секции, нить 17
Конец закрытой секции, нить 17
Начало закрытой секции, нить 23
Конец закрытой секции, нить 23
Начало закрытой секции, нить 22
Конец закрытой секции, нить 22
Начало закрытой секции, нить 1
Конец закрытой секции, нить 1
Начало закрытой секции, нить 5
Конец закрытой секции, нить 5
Начало закрытой секции, нить 21
Конец закрытой секции, нить 21
Начало закрытой секции, нить 11
Конец закрытой секции, нить 11
Начало закрытой секции, нить 19
Конец закрытой секции, нить 19
Начало закрытой секции, нить 8
Конец закрытой секции, нить 8
Начало закрытой секции, нить 9
Конец закрытой секции, нить 9
Начало закрытой секции, нить 0
Конец закрытой секции, нить 0
Начало закрытой секции, нить 7
Конец закрытой секции, нить 7
Начало закрытой секции, нить 3
Конец закрытой секции, нить 3
Начало закрытой секции, нить 12
Конец закрытой секции, нить 12
Начало закрытой секции, нить 2
Конец закрытой секции, нить 2
Начало закрытой секции, нить 6
Конец закрытой секции, нить 6
Начало закрытой секции, нить 20
Конец закрытой секции, нить 20
Начало закрытой секции, нить 13
Конец закрытой секции, нить 13
Начало закрытой секции, нить 14
Конец закрытой секции, нить 14
Начало закрытой секции, нить 4
Конец закрытой секции, нить 4
Начало закрытой секции, нить 16
Конец закрытой секции, нить 16
vasiliy@BEAST:/mnt/d/Dropbox/lol/B
```

Переменная lock используется для блокировки. В последовательной области производится инициализация данной переменной с помощью функции omp_init_lock(). В начале параллельной области каждая нить присваивает переменной n свой порядковый номер. После этого с помощью функции omp_set_lock() одна из нитей выставляет блокировку, а остальные нити ждут, пока нить, вызвавшая эту функцию, не снимет блокировку с помощью функции omp_unset_lock(). Все нити по очереди выведут сообщения "Начало закрытой секции..." и "Конец закрытой секции...", при этом между двумя сообщениями от одной нити не могут встретиться сообщения от другой нити. В конце с помощью функции omp_destroy_lock() происходит освобождение переменной lock.

Пример

```c
#include <stdio.h>
#include <omp.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    omp_set_num_threads(4);
    omp_lock_t lock;
    int n;
    omp_init_lock(&lock);
    #pragma omp parallel private (n)
    {
        n=omp_get_thread_num();
        while (!omp_test_lock (&lock))
        {
            printf("Секция закрыта, нить %d\n", n);
            sleep(2);
        }
        printf("Начало закрытой секции, нить %d\n", n);
        sleep(5);
        printf("Конец закрытой секции, нить %d\n", n);
        omp_unset_lock(&lock);
    }
    omp_destroy_lock(&lock);
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol
Начало закрытой секции, нить 0
Секция закрыта, нить 2
Секция закрыта, нить 1
Секция закрыта, нить 3
Секция закрыта, нить 2
Секция закрыта, нить 1
Секция закрыта, нить 3
Секция закрыта, нить 2
Секция закрыта, нить 1
Секция закрыта, нить 3
Конец закрытой секции, нить 0
Начало закрытой секции, нить 2
Секция закрыта, нить 1
Секция закрыта, нить 3
Секция закрыта, нить 1
Секция закрыта, нить 3
Секция закрыта, нить 1
Секция закрыта, нить 3
Конец закрытой секции, нить 2
Начало закрытой секции, нить 1
Секция закрыта, нить 3
Секция закрыта, нить 3
Секция закрыта, нить 3
Конец закрытой секции, нить 1
Начало закрытой секции, нить 3
Конец закрытой секции, нить 3
vasiliy@BEAST:/mnt/d/Dropbox/lol
```

Пример иллюстрирует применение технологии замков и использование функции omp_test_lock(). В данном примере переменная lock используется для блокировки. В начале производится инициализация данной переменной с помощью функции omp_init_lock(). В параллельной области каждая нить присваивает переменной n свой порядковый номер. После этого с помощью функции omp_test_lock() нити попытаются выставить блокировку. Одна из нитей успешно выставит блокировку, другие же нити напечатают сообщение "Секция закрыта...", приостановят работу на две секунды с помощью функции sleep(), а после снова будут пытаться установить блокировку. Нить, которая установила блокировку, должна снять её с помощью функции omp_unset_lock(). Таким образом, код, находящийся между функциями установки и снятия блокировки, будет выполнен каждой нитью по очереди. В данном случае, все нити по очереди выведут сообщения "Начало закрытой секции..." и "Конец закрытой секции...", но при этом между двумя сообщениями от одной нити могут встретиться сообщения от других нитей о неудачной попытке войти в закрытую секцию. В конце с помощью функции omp_destroy_lock() происходит освобождение переменной lock.

Директива flush

- Поскольку в современных параллельных вычислительных системах может использоваться сложная структура и иерархия памяти, пользователь должен иметь гарантии того, что в необходимые ему моменты времени все нити будут видеть единый согласованный образ памяти. Именно для этих целей и предназначена директива flush.

#pragma omp flush [(список)]

Что?

- Выполнение данной директивы предполагает, что значения всех переменных (или переменных из списка, если он задан), временно хранящиеся в регистрах и кэш-памяти текущей нити, будут занесены в основную память; все изменения переменных, сделанные нитью во время работы, станут видимы остальным нитям; если какая-то информация хранится в буферах вывода, то буферы будут сброшены и т.п. При этом операция производится только с данными вызвавшей нити, данные, изменявшиеся другими нитями, не затрагиваются. Поскольку выполнение данной директивы в полном объёме может повлечь значительных накладных расходов, а в данный момент нужна гарантия согласованного представления не всех, а лишь отдельных переменных, то эти переменные можно явно перечислить в директиве списком. До полного завершения операции никакие действия с перечисленными в ней переменными не могут начаться.

Что?

- Неявно flush без параметров присутствует в директиве barrier, на входе и выходе областей действия директив parallel, critical, ordered, на выходе областей распределения работ, если не используется опция nowait, в вызовах функций omp_set_lock(), omp_unset_lock(), omp_test_lock(), omp_set_nest_lock(), omp_unset_nest_lock(), omp_test_nest_lock(), если при этом замок устанавливается или снимается, а также перед порождением и после завершения любой задачи (task). Кроме того, flush вызывается для переменной, участвующей в операции, ассоциированной с директивой atomic. Заметим, что flush не применяется на входе области распределения работ, а также на входе и выходе области действия директивы master.

Дополнительные переменные среды и функции

- Переменная OMP_MAX_ACTIVE_LEVELS задаёт максимально допустимое количество вложенных параллельных областей. Значение может быть установлено при помощи вызова функции omp_set_max_active_levels().

```
void omp_set_max_active_levels(int max);
```

- Если значение max превышает максимально допустимое в системе, будет установлено максимально допустимое в системе значение. При вызове из параллельной области результат выполнения зависит от реализации.
- Значение переменной OMP_MAX_ACTIVE_LEVELS может быть получено при помощи вызова функции omp_get_max_active_levels().

```
int omp_get_max_active_levels(void);
```

Дополнительные переменные среды и функции

- Функция omp_get_level() выдаёт для вызвавшей нити количество вложенных параллельных областей в данном месте кода.

```
int omp_get_level(void);
```

- При вызове из последовательной области функция возвращает значение 0.
- Функция omp_get_ancestor_thread_num() возвращает для уровня вложенности параллельных областей, заданного параметром level, номер нити, породившей данную нить.

```
int omp_get_ancestor_thread_num(int level);
```

Дополнительные переменные среды и функции

- Если level меньше нуля или больше текущего уровня вложенности, возвращается -1. Если level=0, функция вернёт 0, а если level=omp_get_level(), вызов эквивалентен вызову функции omp_get_thread_num().
- Функция omp_get_team_size() возвращает для заданного параметром level уровня вложенности параллельных областей количество нитей, порождённых одной родительской нитью.

```
int omp_get_active_level(void);
```

- При вызове из последовательной области возвращает значение 0.

Дополнительные переменные среды и функции

- Переменная среды OMP_STACKSIZE задаёт размер стека для создаваемых из программы нитей. Значение переменной может задаваться в виде size | sizeB | sizeK | sizeM | sizeG, где size – положительное целое число, а буквы B, K, M, G задают соответственно, байты, килобайты, мегабайты и гигабайты. Если ни одной из этих букв не указано, размер задаётся в килобайтах. Если задан неправильный формат или невозможно выделить запрошенный размер стека, результат будет зависеть от реализации.
- Например, в Linux в командной оболочке bash задать размер стека можно при помощи следующей команды:

```
export OMP_STACKSIZE=2000K
```

Дополнительные переменные среды и функции

- Переменная среды OMP_WAIT_POLICY задаёт поведение ждущих процессов. Если задано значение ACTIVE, то ждущему процессу будут выделяться циклы процессорного времени, а при значении PASSIVE ждущий процесс может быть отправлен в спящий режим, при этом процессор может быть назначен другим процессам.
- Переменная среды OMP_THREAD_LIMIT задаёт максимальное число нитей, допустимых в программе. Если значение переменной не является положительным целым числом или превышает максимально допустимое в системе число процессов, поведение программы будет зависеть от реализации. Значение переменной может быть получено при помощи процедуры omp_get_thread_limit().

```
int omp_get_thread_limit(void);
```

Заметки

- Если целевая вычислительная платформа является многопроцессорной и/или многоядерной, то для повышения быстродействия программы нужно задействовать все доступные пользователю вычислительные ядра. Чаще всего разумно порождать по одной нити на вычислительное ядро, хотя это не является обязательным требованием. Например, для первоначальной отладки может быть вполне достаточно одноядерного процессора, на котором порождается несколько нитей, работающих в режиме разделения времени. Порождение и уничтожение нитей в OpenMP являются относительно недорогими операциями, однако надо помнить, что многократное совершение этих действий (например, в цикле) может повлечь существенное увеличение времени работы программы.

Заметки

- Для того чтобы получить параллельную версию, сначала необходимо определить ресурс параллелизма программы, то есть, найти в ней участки, которые могут выполняться независимо разными нитями. Если таких участков относительно немного, то для распараллеливания чаще всего используются конструкции, задающие конечный (неитеративный) параллелизм, например, параллельные секции, конструкция workshare или низкоуровневое распараллеливание по номеру нити.

Заметки

- Однако, как показывает практика, наибольший ресурс параллелизма в программах сосредоточен в циклах. Поэтому наиболее распространенным способом распараллеливания является то или иное распределение итераций циклов. Если между итерациями некоторого цикла нет информационных зависимостей, то их можно каким-либо способом раздать разным процессорам для одновременного исполнения. Различные способы распределения итераций позволяют добиваться максимально равномерной загрузки нитей, между которыми распределяются итерации цикла.

Заметки

- Статический способ распределения итераций позволяет уже в момент написания программы точно определить, какой нити достанутся какие итерации. Однако он не учитывает текущей загруженности процессоров, соотношения времён выполнения различных итераций и некоторых других факторов. Эти факторы в той или иной степени учитываются динамическими способами распределения итераций. Кроме того, возможно отложить решение по способу распределения итераций на время выполнения программы (например, выбирать его, исходя из текущей загруженности нитей) или возложить выбор распределения на компилятор и/или систему выполнения.

Заметки

- Обмен данными в OpenMP происходит через общие переменные. Это приводит к необходимости разграничения одновременного доступа разных нитей к общим данным. Для этого предусмотрены достаточно развитые средства синхронизации. При этом нужно учитывать, что использование излишних синхронизаций может существенно замедлить программу.

Заметки

- Использование идеи инкрементального распараллеливания позволяет при помощи OpenMP быстро получить параллельный вариант программы. Взяв за основу последовательный код, пользователь шаг за шагом добавляет новые директивы, описывающие новые параллельные области. Нет необходимости сразу распараллеливать всю программу, её создание ведется последовательно, что упрощает и процесс программирования, и отладку.

Заметки

- Программа, созданная с использованием технологии OpenMP, может быть использована и в качестве последовательной программы. Таким образом, нет необходимости поддерживать последовательную и параллельную версии. Директивы OpenMP просто игнорируются последовательным компилятором, а для вызова функций OpenMP могут быть подставлены специальные «заглушки» (stubs), текст которых приведен в описании стандарта. Они гарантируют корректную работу программы в последовательном случае – нужно только перекомпилировать программу и подключить другую библиотеку.

Заметки

- Одним из достоинств OpenMP его разработчики считают поддержку так называемых оторванных (orphaned) директив. Это предполагает, что директивы синхронизации и распределения работы могут не входить непосредственно в лексический контекст параллельной области. Например, можно вставлять директивы в вызываемую подпрограмму, предполагая, что её вызов произойдёт из параллельной области.

Пример “orphaned” директивы

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#define VECLEN 100

float a[VECLEN], b[VECLEN], sum;

float dotprod ()
{
int i,tid;

tid = omp_get_thread_num();
#pragma omp for reduction(+:sum)
  for (i=0; i < VECLEN; i++)
    {
    sum = sum + (a[i]*b[i]);
    printf("  tid= %d i=%d\n",tid,i);
    }
}
```

```
int main (int argc, char *argv[]) {
int i;

for (i=0; i < VECLEN; i++)
  a[i] = b[i] = 1.0 * i;
sum = 0.0;

#pragma omp parallel
  dotprod();

printf("Sum = %f\n",sum);

}
```

Заметки

- OpenMP может использоваться совместно с другими технологиями параллельного программирования, например, с MPI. Обычно в этом случае MPI используется для распределения работы между несколькими вычислительными узлами, а OpenMP затем используется для распараллеливания на одном узле.

Хватит на сегодня. Спасибо за внимание!

Разработчик: Это много времени не займет. Я могу просто использовать код из другого проекта.

Результат:

